

**Instituto Tecnológico de Costa Rica**  
**Escuela de Ingeniería en Computadores**  
**Curso: CE-1101 Introducción a la Programación**

Profesor: Ellioth Alberto Ramírez Trejos

**DOCUMENTO TÉCNICO**

*Proyecto 1*

Estudiante: Randall Mora Zamora

Carné: 2023344786

Fecha: Abril 2026

Versión: 1.0

## Índice

---

|                                              |    |
|----------------------------------------------|----|
| Índice                                       | 2  |
| Palabras clave y Acrónimos                   | 3  |
| 1. Introducción                              | 4  |
| 2. Marco Teórico                             | 5  |
| 3. Descripción del Programa                  | 6  |
| 3.1 Diagrama de flujo general del sistema    | 6  |
| 4. Resultados Obtenidos                      | 7  |
| 4.1 Descripción de las Funciones Principales | 7  |
| 4.2 Descripción de la Interfaz Gráfica       | 8  |
| 5. Conclusiones y Recomendaciones            | 9  |
| 5.1 Conclusiones                             | 9  |
| 5.2 Recomendaciones                          | 9  |
| Bibliografía                                 | 10 |

## Palabras clave y Acrónimos

**Palabras clave:** juego de batalla por turnos, Python, Tkinter, recursividad, animación por frames, interfaz gráfica de usuario, música, MCI, sprites.

La siguiente tabla presenta los acrónimos y términos técnicos empleados a lo largo del documento:

| Término        | Definición                                                                  |
|----------------|-----------------------------------------------------------------------------|
| <b>GUI</b>     | Graphical User Interface / Interfaz Gráfica de Usuario                      |
| <b>HP</b>      | Hit Points / Puntos de Vida del personaje                                   |
| <b>ATK</b>     | Attack / Puntos de Ataque del personaje                                     |
| <b>DEF</b>     | Defense / Puntos de Defensa del personaje                                   |
| <b>KO</b>      | Knock Out / Estado de derrota de un personaje                               |
| <b>MCI</b>     | Media Control Interface / Interfaz de Control de Medios de Windows          |
| <b>DXA</b>     | Document eXtended Attribute / Unidad de medida interna de documentos Office |
| <b>TEC</b>     | Tecnológico de Costa Rica                                                   |
| <b>PIL</b>     | Python Imaging Library / Biblioteca de procesamiento de imágenes            |
| <b>PNG</b>     | Portable Network Graphics / Formato de imagen sin pérdida                   |
| <b>MP3</b>     | MPEG Audio Layer III / Formato de audio comprimido                          |
| <b>after()</b> | Método de Tkinter para programar llamadas diferidas                         |

## 1. Introducción

---

El presente documento técnico describe el diseño, desarrollo e implementación del proyecto Disney's Epic Adventure, un juego de batallas por turnos desarrollado en Python como parte del curso CE-1101 Introducción a la Programación del Instituto Tecnológico de Costa Rica.

El sistema simula un juego de rol por turnos de estilo similar a Pokémon o Final Fantasy, en el que el jugador asume el rol de Guardián de las Historias (Story Keeper) y debe recorrer cinco zonas del mundo para derrotar a los Huecos (Hollows), entidades que capturaron a personajes de franquicias animadas populares dentro de Efigies.

El objetivo principal del proyecto es demostrar el dominio de los conceptos fundamentales de programación: estructuras de datos, funciones recursivas, manejo de archivos, interfaces gráficas con Tkinter y animaciones implementadas. El requisito de recursividad permea todas las operaciones del sistema, desde la lectura de archivos hasta la lógica del combate y la reproducción de música.

Este documento cubre la arquitectura general del sistema, las decisiones de diseño técnico, la descripción de las funciones principales, la interfaz gráfica y las conclusiones del proceso de desarrollo.

## 2. Marco Teórico

---

El sistema fue desarrollado íntegramente en el lenguaje de programación Python 3, seleccionado por su legibilidad, su amplio ecosistema de bibliotecas y por ser el lenguaje indicado en los requerimientos del curso [1].

### 2.1 Recursividad

La recursividad es la técnica mediante la cual una función se llama a sí misma para resolver un problema dividiéndolo en subproblemas más simples. En este proyecto, la recursividad es obligatoria en todas las operaciones iterativas: lectura de archivos, carga de imágenes, lógica de combate, generación de equipos y construcción de la interfaz gráfica [2].

### 2.2 Tkinter

Tkinter es la biblioteca estándar de Python para la creación de interfaces gráficas de usuario (GUI). Proporciona widgets como Frame, Label, Button, Canvas y Entry, que se utilizan para construir las distintas pantallas del juego. La dinámica de pantallas se implementa alternando `pack()` y `pack_forget()` sobre los frames, sin necesidad de múltiples ventanas [3].

### 2.3 Animación con `after()`

La animación de sprites se logra mediante el método `ventana.after(ms, función)`, que programa la ejecución diferida de una función sin bloquear la interfaz. Esto permite actualizar el frame visible de un personaje a intervalos regulares, creando la ilusión de movimiento de forma recursiva y sin utilizar hilos ni bucles bloqueantes [3].

### 2.4 PIL / Pillow

La biblioteca Pillow (fork de PIL) se utiliza para abrir, redimensionar, voltear y convertir imágenes PNG al formato `Photomage` compatible con Tkinter. Es esencial para cargar los sprites de los personajes, los avatares del jugador y las imágenes de perfil de los Hollows [4].

### 2.5 Windows MCI (`winmm.dll`)

La reproducción de música MP3 se realiza mediante la interfaz MCI (Media Control Interface) de Windows, accedida a través del módulo `ctypes` de Python. Esta interfaz permite abrir y reproducir archivos de audio sin dependencias externas adicionales. El bucle de música se implementa con `ventana.after()`, comprobando recursivamente si la canción terminó para reiniciarla [5].

### 2.6 Juegos de batalla por turnos

El modelo de juego por turnos (como Pokémon o Final Fantasy) enfrenta a dos bandos alternando acciones hasta que uno queda sin personajes activos. En este proyecto, el jugador elige entre tres acciones (Ataque, Ataque Fuerte y Defensa) mientras el Hollow actúa de forma

completamente aleatoria. El daño se calcula con la fórmula:  $\text{Daño} = \text{ATK del atacante} - \text{DEF del defensor}$ , con un mínimo garantizado de 1 punto [6].

### 3. Descripción del Programa

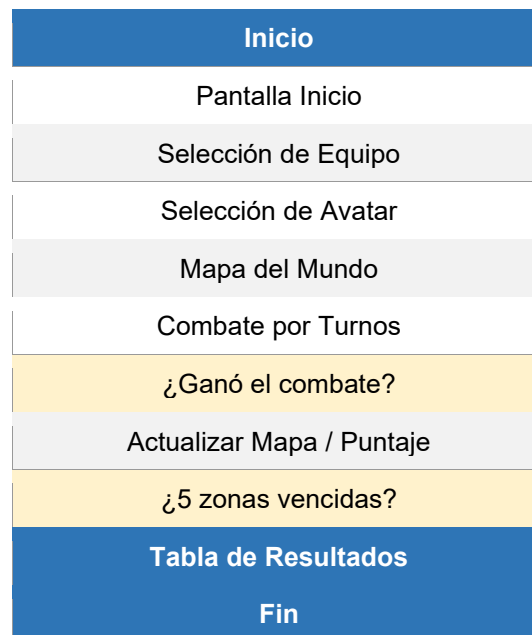
El programa se estructura en un único archivo principal (mai.py) que contiene toda la lógica del juego, la carga de recursos y la construcción de la interfaz. Los datos de los personajes se leen de un archivo de texto externo (Personajes.txt) y los recursos gráficos y de audio se organizan en carpetas separadas.

El flujo general de ejecución sigue la siguiente secuencia de pantallas:

- Pantalla de Inicio: el jugador ingresa su nombre y presiona INICIAR.
- Selección de Equipo: elige 3 personajes de un catálogo de 15 tarjetas animadas.
- Selección de Avatar: elige su representante visual de entre 3 opciones.
- Mapa del Mundo: navega entre las 5 zonas disponibles para retarlas.
- Combate por Turnos: enfrenta al Hollow de la zona elegida en batallas uno a uno.
- Pantalla de Resultados: muestra la tabla de puntajes acumulados al finalizar.

#### 3.1 Diagrama de flujo general del sistema

El siguiente diagrama resume el flujo principal de navegación del sistema:



## 4. Resultados Obtenidos

El sistema implementado cumple con la totalidad de los requerimientos definidos en el enunciado del proyecto. A continuación se detalla la descripción técnica de las funciones principales y de la interfaz gráfica.

### 4.1 Descripción de las Funciones Principales

La siguiente tabla describe las funciones más importantes del sistema, indicando sus entradas, salidas y descripción de funcionamiento:

| Función                      | Entradas                                  | Salidas             | Descripción                                                                                                                                                  |
|------------------------------|-------------------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| leer_personajes()            | Ruta del archivo<br>Personajes.txt        | Lista de tuplas     | Lee el archivo de personajes recursivamente línea por línea y parsea sus atributos (nombre, HP, ATK, DEF, carpeta, rangos de frames).                        |
| cargar_frames_accion()       | Carpeta, lista de frames, tamaño, flip    | Lista de PhotoImage | Carga recursivamente los frames PNG de una acción (caminar, ataque, morir). Si flip=True, voltea la imagen para el enemigo.                                  |
| animar()                     | Estado de animación, label, intervalo     | —                   | Avanza un frame y se reprograma con ventana.after() para crear el bucle de animación.                                                                        |
| turno(tipo)                  | Tipo de acción: normal / fuerte / defensa | —                   | Ejecuta el turno completo: calcula daño del jugador, aplica modificadores de defensa, luego la IA elige y ejecuta su acción. Captura al personaje derrotado. |
| iniciar_combate(pos)         | Número de zona (1–5)                      | —                   | Prepara la pantalla de combate: actualiza perfiles, asigna el equipo del hollow de esa zona y lanza la primera ronda.                                        |
| iniciar_ronda()              | —                                         | —                   | Selecciona el personaje activo de cada bando. Llama a fin_juego() si algún equipo quedó sin personajes vivos.                                                |
| elegir_accion_ai_aleatoria() | —                                         | String de acción    | Genera aleatoriedad usando los microsegundos del reloj del sistema (sin módulo random). Elige entre normal, fuerte o defensa.                                |
| calcular_dano(atk, def)      | ATK del atacante, DEF del defensor        | Entero $\geq 1$     | Fórmula del proyecto: $\text{daño} = \text{ATK} - \text{DEF}$ ; si el resultado es $\leq 0$ , retorna 1 (daño mínimo garantizado).                           |



| Función                  | Entradas                          | Salidas                        | Descripción                                                                                                                                        |
|--------------------------|-----------------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| reproducir_musica(clave) | Clave: 'inicio', 'mapa' o int 1–5 | —                              | Abre el MP3 via Windows MCI (winmm.dll). El bucle infinito se implementa con ventana.after() que verifica si la canción terminó cada 500 ms.       |
| generar_hollows()        | —                                 | Lista de 5 grupos de 3 nombres | Mezcla los 15 personajes con un algoritmo Fisher-Yates recursivo usando el timestamp como semilla y los agrupa en 5 equipos de 3.                  |
| fin_juego(gano)          | Booleano: True = victoria         | —                              | Maneja el fin del combate: suma puntos, bloquea la zona vencida, detiene la música de combate y transiciona al mapa o a la pantalla de resultados. |
| reiniciar()              | —                                 | —                              | Resetea todo el estado global y vuelve a la pantalla de inicio. Reactiva botones del mapa y de selección de personajes.                            |

## 4.2 Descripción de la Interfaz Gráfica

La interfaz gráfica fue diseñada con Tkinter siguiendo un esquema de pantallas únicas (single-window multi-frame): todos los frames conviven en la misma ventana (1100 × 720 px) y se alternan mediante pack() / pack\_forget(). Se aplicó una paleta de colores oscura con acentos azules para mantener coherencia visual en todas las pantallas.

Las seis pantallas principales son:

- Pantalla de Inicio: fondo oscuro (#0d0d1a), imagen de título cargada desde Fondos/Titulo.png, campo de texto para el nombre y botones INICIAR y About.
- Selección de Equipo: grilla scrollable de 5 columnas con tarjetas animadas (120 × 120 px). Panel lateral que muestra los 3 slots del equipo y permite deseleccionar personajes con clic.
- Selección de Avatar: tres cards con imágenes de 180 × 180 px. La card seleccionada se resalta en verde. El botón EMPEZAR se habilita al elegir un avatar.
- Mapa del Mundo: canvas con la imagen del mapa (860 × 550 px) y 5 botones posicionados con place() sobre ella. Las zonas vencidas quedan bloqueadas en rojo.
- Pantalla de Combate: barra superior de perfiles (avatar del jugador a la izquierda, imagen del Hollow a la derecha), sprites animados de 320 × 320 px para cada personaje activo, barras de vida codificadas por color (verde / amarillo / rojo) y botones de acción (Ataque, Ataque Fuerte, Defensa). En la parte inferior aparecen los botones del equipo del jugador que permiten cambiar de personaje.
- Tabla de Resultados: muestra el historial de partidas de la sesión con nombre, puntaje y resultado. Incluye botones para volver al menú o salir.

El sistema de música utiliza Windows MCI para reproducir los archivos MP3 sin bibliotecas externas. Se reproducen canciones diferenciadas para el menú, el mapa y cada zona de combate (5 canciones distintas), con transiciones automáticas entre pantallas.

## 5. Conclusiones y Recomendaciones

---

### 5.1 Conclusiones

- Se logró implementar un juego de batalla por turnos completamente funcional en Python utilizando Tkinter como único framework de interfaz gráfica, cumpliendo con todos los requerimientos del proyecto.
- El logro del uso de recursividad en todos los niveles del sistema, desde la lectura de archivos hasta la lógica de combate y la animación de sprites, reforzando el concepto en la práctica.
- La animación de personajes mediante `ventana.after()` demostró ser un patrón eficaz para crear bucles de animación no bloqueantes dentro de una aplicación Tkinter, sin necesidad de hilos adicionales.
- La reproducción de música con Windows MCI (ctypes) permitió integrar audio MP3 sin instalar dependencias externas, aunque con la limitación de funcionar exclusivamente en sistemas Windows.
- La separación de datos en archivos de texto externos (`Personajes.txt`, `Avatares.txt`) facilita la extensión del juego: agregar nuevos personajes no requiere modificar el código fuente.

### 5.2 Recomendaciones

- La pantalla de combate podría enriquecerse con efectos visuales de daño (números flotantes, parpadeo del sprite golpeado) para mejorar la retroalimentación al jugador.
- La generación aleatoria de equipos Hollow podría mejorarse usando una función de hash más robusta, aunque para fines académicos el uso del timestamp es suficiente.

## Bibliografía

---

Las siguientes referencias, todas en idioma español, se citan en formato IEEE:

- [1]** E. Matthes, *Aprende Python: Una introducción práctica y sin esfuerzo a la programación*, 2.ª ed. Madrid, España: Anaya Multimedia, 2020.
- [2]** M. Pilgrim, *Inmersión en Python 3*, trad. por la comunidad Python Hispano. [En línea]. Disponible: <https://diveintopython3.problemsolving.io/>. [Accedido: 6 abr. 2026].
- [3]** Python en Español, "Documentación oficial de Python 3 en español," docs.python.org, 2024. [En línea]. Disponible: <https://docs.python.org/es/3/>. [Accedido: 6 abr. 2026].
- [4]** Python en Español, "tkinter — Interfaz de Python con Tcl/Tk," docs.python.org, 2024. [En línea]. Disponible: <https://docs.python.org/es/3/library/tkinter.html>. [Accedido: 7 abr. 2026].
- [5]** Pillow Contributors, "Documentación de Pillow (PIL Fork)," Read the Docs, 2024. [En línea]. Disponible: <https://pillow.readthedocs.io/en/stable/>. [Accedido: 12 abr. 2026].
- [6]** Instituto Tecnológico de Costa Rica, "Enunciado Proyecto 1 — Disney's Epic Adventure," CE-1101 Introducción a la Programación, I Semestre 2026. Inicio del proyecto: 6 de abril de 2026.